Ad Soyad: _____	**Tarih:** _____
Bölüm 13 – Ünite 2 – *OBYS İstemci Tasarımı – Çalışma Kağıdı*	

Süre

Bu çalışma kağıdını tamamlamak için tahmini süre: **35–45 dakika**.

Konu Hatırlatma

Bu ünitede asenkron OBYS istemcisinin **durum makinesi**, **kimlik doğrulama akışı** (4 adım), **komut/cevap deseni**, **korelasyon ID** ve **yeniden bağlanma stratejileri** işlendi. Detaylar için: `sec:13_u2_baglanti_kurulumu`, `sec:13_u2_kimlik_dogrulama`, `sec:13_u2_komut_gonderme`.

Görev 1: İstemci Durum Makinesi – Geçişleri Yaz

(7 dakika – 20 puan)

G1: Durumlar Arasındaki Tetikleyici Olayları Doldur

ServerConnection altı durumda (`BagliDegil`, `Baglaniyor`, `Bagli`, `KimlikDogrulaniyor`, `Hazir`, `YenidenBaglaniyor`) yaşar. Her geçiş için *tetikleyici olayı / sinyali* yazın.

Bu durumdan	→	Bu duruma	Tetikleyici (sinyal/olay)
BagliDegil	→	Baglaniyor	_____
Baglaniyor	→	Bagli	_____
Bagli	→	KimlikDogrulaniyor	_____
KimlikDogrulaniyor	→	Hazir	_____
Hazir	→	YenidenBaglaniyor	_____
YenidenBaglaniyor	→	Baglaniyor	_____

Soru: Hangi geçiş(ler) GUI'ye `durumDegisti` sinyali emit etmek için *kritik*tir? Niçin?

_____

_____

Görev 2: Kimlik Doğrulama – 4 Adımı Sırala

(5 dakika – 14 puan)

G2: Aşağıdaki Adımları Doğru Sırayla Numaralayın

Karışık verilen 4 adımı, kimlik doğrulama akışındaki sıralarına göre 1-2-3-4 olarak numaralandırın.

_____ Sunucu veritabanından kullanıcının salt'ını ve depolanmış hash'ini çeker; istemcinin gönderdiği hash ile sabit zamanlı karşılaştırma yapar.

_____ Sunucu `LOGIN_OK { token, expiry, role }` veya `LOGIN_FAIL { reason }` cevabını döndürür.

_____ Kullanıcı UI'de kullanıcı adı ve parola girer; "Giriş" butonuna basar; *ServerConnection::login* slot'u çağrılır.

_____ İstemci length-prefix çerçeveli JSON `LOGIN` mesajını gönderir (içinde `kullanici_adi` ve `parola_hash`).

Soru: İstemci ham parolayı sunucuya neden *göndermez*? Bir cümlede özetleyin:

_____

_____

Görev 3: `pending_requests_` Haritasını Simüle Et (10 dakika – 25 puan)

G3: 3 Paralel İstek Senaryosunu Elle Yürüt

İstemci, t=0 anında üç istek gönderiyor. Sunucu cevapları farklı sırada döndürür. Her satır için `pending_requests_` haritasının *o andaki* içeriğini yazın.

Atılan istekler (sırayla):

- `id=101`, komut `ogrenciListele`, callback `cb_listele`
- `id=102`, komut `notGetir(12345)`, callback `cb_not`
- `id=103`, komut `transkript(12345)`, callback `cb_trk`

Sunucudan dönen cevap sırası: önce `id=102`, sonra `id=103`, en son `id=101`.

Aşama	Bu anda `pending_requests_` haritası
3 istek de gönderildi, hiç cevap yok	
`id=102` cevabı geldi, callback çağrıldı	
`id=103` cevabı geldi, callback çağrıldı	
`id=101` cevabı geldi, callback çağrıldı	

Soru: Eğer `id=101` cevabı *hiç gelmezse* ne olur? Bu durum bellekte ne yaratır ve nasıl önlenir?

Görev 4: Asenkron Hata Callback'i – GUI Bilgilendirme (8 dakika – 22 puan)

G4: Hata Yolunu Tasarla

Sunucudan `LOGIN_FAIL` dönerse, GUI'deki **LoginPenceresi** kullanıcıya hata göstermeli. Aşağıdaki kalıbı doldurun ve hangi *Qt mekanizmasının* GUI thread'ini güvenli güncellediğini açıklayın.

```
void ServerConnection::cevapIsle(const QJsonObject& cevap) {
    quint64 id = cevap.value("id").toVariant().toULongLong();
    auto it = pending_requests_.find(id);
    if (it == pending_requests_.end()) return;    // Bilinmeyen ID

    auto cb = std::move(it->second);
    pending_requests_.erase(it);

    // Hata mi, basari mi?
    if (cevap.value("durum").toString() == "hata") {
        // (1) Hata yolunu emit et:
        emit hataOlustu(cevap.value("mesaj").toString());
        // (2) Callback'i de hata bilgisiyle cagir:
        cb(cevap);    // cb kendisi durumu kontrol etmeli
    } else {
        cb(cevap);
    }
}
```

Soru a: `ServerConnection` bir worker thread'inde, **LoginPenceresi** ise GUI thread'inde. `emit hataOlustu(...)` sinyalinin GUI'ye *güvenli ulaşması* için Qt hangi mekanizmayı otomatik kullanır?

Soru b: Eğer `Qt::DirectConnection` açıkça ayarlasaydık ne tehlike olurdu?

Soru c: GUI tarafında bu sinyali *tek satır kod* ile **LoginPenceresi::hataGoster** slot'una bağlayın:

```
connect(____________________________________________________________);
```

İpucu

`Qt::AutoConnection`: aynı thread'deyse `DirectConnection`, farklı thread'deyse `QueuedConnection` olur. `QueuedConnection` hedef thread'in event loop'una mesaj olarak gönderir.

Görev 5: BONUS – Exponential Backoff Tablosu

(7 dakika – 19 puan)

G5: Yeniden Bağlanma Beklemelerini Hesapla

Bağlantı koparsa istemci hemen yeniden denemez; *exponential backoff* uygular. Başlangıç gecikme 1 saniye; her başarısız denemeden sonra 2 katına çıkar; üst sınır 60 saniye.

Aşağıdaki tabloyu doldurun:

Deneme	Bekleme süresi (sn)	Toplam geçen süre (sn)
1 (ilk kopma)	_____	_____
2	_____	_____
3	_____	_____
4	_____	_____
5	_____	_____
6	_____	_____
7	_____	_____

Soru a: 7. denemeden sonra üst sınıra ulaşılmış mıdır? Üst sınıra ulaşılınca ne yapılmalı?

Soru b: Tüm istemciler aynı anda kopup aynı backoff'u uygularsa hangi sorun (*thundering herd*) doğar? Çözüm önerisi (jitter) tek cümle:

Tartışma Soruları (sınıf içi açık uçlu)

T1. Korelasyon ID üretmek için `atomic counter` mı yoksa `UUID v4` mü daha uygun? OBYS gibi tek istemcili (single-process) bir uygulamada hangisinin maliyeti gereksiz?

T2. "Senkron çağrı UI thread'inde yasaktır" kuralı geçerlidir, ama `waitForConnected` bazen kabul edilebilir mi? Hangi durumda?

T3. GUI'nin ağdan habersiz olması mimari olarak değerli; ama *tamamen* habersiz olabilir mi? Bağlantı kopması göstergesi nasıl tasarlanır?

T4. Oturum jetonu (token) süresi dolmuşsa istemci ne yapmalı? Kullanıcıdan tekrar parola istemek mi, sessizce yenilemek mi? Avantaj/dezavantajları?